# Universidad Nacional Autónoma de México

**Facultad de Estudios Superiores Acatlán**







# Database Administrator

## Empresa - Proyecto

*Query Teach Solutions*

*Movilidad: Ecobici CDMX*

**Integrantes**

Estrada Romero Meliza Edith

Ricardo López Ramírez

Ruiz Sánchez Emiliano

Cabrera Trejo José Luis

Actualización: 25 de octubre de 2026

# Índice


**1. Tema y Descripción del Proyecto** **1**
- 1.1. Descripción General . . . 1
- 1.2. Elección del Tema . . . 1
- 1.3. Información a Visualizar en la Base de Datos . . . 1
- 1.4. Finalidad Principal . . . 1

**2. Diagrama Entidad-Relación** **2**

**3. Enlace a Repositorio GitHub.** **3**

**4. Enlace Google Drive** **3**

**5. Estimación Teórica y Proyección de Espacio en Disco.** **4**
- 5.1. 1. Tamaño por tipo de dato y fila (Row Size) . . . 4
- 5.2. 2. Cálculo del Espacio de Carga Inicial . . . 4
- 5.3. 3. Simulación y Proyección de Crecimiento . . . 4

**6. Consulta al Diccionario de Datos y Comparación de Almacenamiento.** **5**
- 6.1. Justificación Teórica . . . 5
- 6.2. Desarrollo de Consultas SQL . . . 5
- 6.3. Matriz de Comparación . . . 6

**7. Alta Disponibilidad, Replicación y Casos de Failover.** **7**
- 7.1. Servidores Primario, Réplica y Balanceador usando Docker. . . . 7
  - 7.1.1. Creando el entorno . . . 7
  - 7.1.2. Visual Studio Code . . . 7
  - 7.1.3. Crea el archivo del Balanceador de Carga . . . 7
  - 7.1.4. Iniciar los servidores . . . 7
- 7.2. Configuración de Permisos para HAProxy (Health Checks) . . . 9
- 7.3. Configuración de Replicación Maestro-Esclavo (GTID) . . . 10
  - 7.3.1. Habilitar GTID y Replicación (Inyección vía Docker Compose) . . . 10
  - 7.3.2. Ejecución de la Arquitectura Limpia . . . 10
  - 7.3.3. Comandos de Replicación SQL . . . 10
  - 7.3.4. Resolución de Incidentes: Error 1396 y Conflicto GTID . . . 11
- 7.4. Cargar base de datos ECOBICI . . . 12
  - 7.4.1. Copiar los archivos CSV al contenedor Linux . . . 12
  - 7.4.2. Mover y modificar el script SQL . . . 12
  - 7.4.3. Inyectar la base de datos . . . 12

**8. Crear Cuenta Google Cloud** **13**
- 8.1. Crear Instancias . . . 14
  - 8.1.1. Regla de Firewall adicional . . . 16
  - 8.1.2. Maquina Virtual . . . 18
  - 8.1.3. Instalar Docker y Docker Compose . . . 18
  - 8.1.4. Dar permisos a tu usuario . . . 18
  - 8.1.5. Crear la estructura del proyecto . . . 18
  - 8.1.6. Instalacion editor de texto nano . . . 18
  - 8.1.7. Crear docker-compose.yml . . . 19
  - 8.1.8. Crear haproxy.cfg . . . 19
  - 8.1.9. Arrancar la infraestructura en la nube . . . 19
  - 8.1.10. Configuración de Permisos para HAProxy (Health Checks) . . . 19
  - 8.1.11. Replicacion GTID . . . 20
  - 8.1.12. Crear el usuario de clonación en el Maestro . . . 20
  - 8.1.13. Comprobar la conexión . . . 20

**9. Poblar la Base de Datos** **21**
9.1. Cargar Data en la Maquina Virtual GCP . . . . . 22
9.1.1. Copiar dentro del contenedor Maestro . . . . . 22
9.1.2. Ejecución del Script en MySQL Workbench . . . . . 23
9.1.3. PRIMARY KEY . . . . . 23
9.1.4. FOREIGN KEY . . . . . 23

# 1. Tema y Descripción del Proyecto

## 1.1. Descripción General

**Movilidad Urbana: Análisis del Sistema Ecobici en la Ciudad de México**, consiste en el diseño, estructuración e implementación de una base de datos relacional analítica enfocada en la red de transporte de bicicletas de la CDMX. Un diseño de arquitectura basada en un Modelo de Estrella, el proyecto procesa y normaliza un volumen masivo de datos transaccionales correspondientes a los años 2024, 2025 y 2026.

## 1.2. Elección del Tema

Se seleccionó este tema porque el sistema Ecobici es fundamental de la movilidad sustentable y el desarrollo urbano en la capital del país. La Data proporcionada por Ecobici CDMX, genera un volumen de información masivo diariamente. Esto nos presentó un escenario ideal, para aplicar técnicas avanzadas de administración de bases de datos. Resolver las limitantes de particionamiento del motor de bases de datos mientras se garantizaba un 100 % de integridad referencial, demuestra el nivel técnico requerido para gestionar arquitecturas de datos.

## 1.3. Información a Visualizar en la Base de Datos

La base de datos está estructurada para visualizar la interacción entre los usuarios y el sistema de movilidad mediante las siguientes dimensiones y métricas:

- **Hechos de Movilidad:** Más de 6.5 millones de registros operativos en la tabla principal y más de 30 millones en el histórico, detallando cada viaje realizado.
- **Perfiles Demográficos:** Género y edad de los usuarios, permitiendo segmentar el comportamiento del consumidor.
- **Dimensión Espacial:** Catálogo de estaciones de retiro y arribo, incluyendo su nomenclatura y zonificación, para trazar las rutas urbanas.
- **Dimensión Temporal:** Fechas exactas y franjas horarias (Mañana, Tarde, Noche, Madrugada) para identificar horas pico y valles de demanda.
- **Inventario Físico:** Estatus y tipo de las bicicletas utilizadas.

Tabla 1: Volumetría de Datos del Proyecto

| **Tabla / Entidad** | **Tipo en Modelo** | **Volumen Aproximado** |
|---|---|---|
| Historico Viajes | Tabla Particionada | $> 30,000,000$ registros |
| Viajes (2025-2026) | Tabla de Hechos | $> 6,500,000$ registros |
| Estaciones | Catálogo / Dimensión | 683 estaciones únicas |
| Tiempos | Dimensión Temporal | 86,400 segundos |

## 1.4. Finalidad Principal

La finalidad principal de esta base de datos no es únicamente el almacenamiento, es servir como un repositorio analítico. El objetivo es transformar los datos en información estratégica que permita a la toma de decisiones optimizar la logística del sistema. Mediante las sentencias DML, la base de datos es capaz de responder preguntas de negocio en segundos. Ejemplo: la saturación de las estaciones por franjas horarias, los patrones de ruta más frecuentes según el género del usuario.

## 2. Diagrama Entidad-Relación

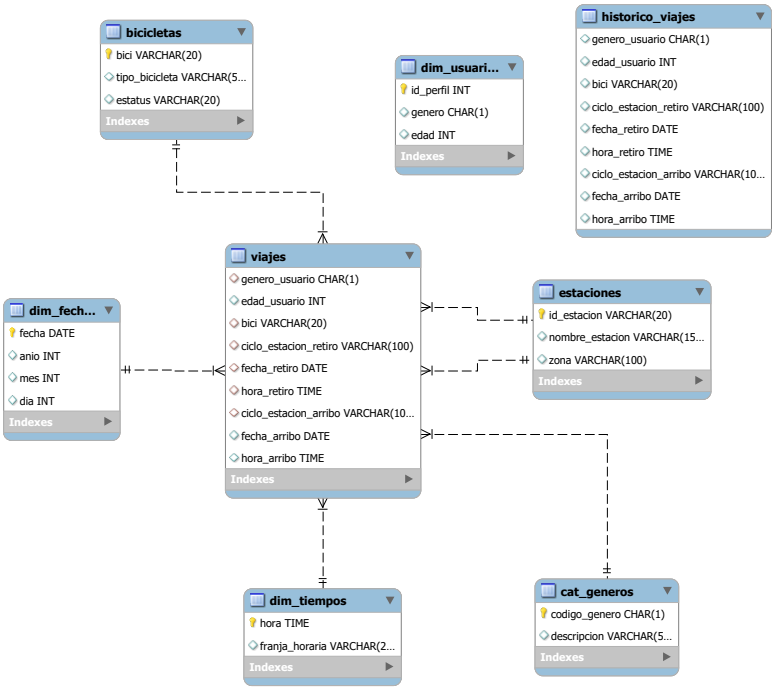



Figura 1: Diagrama Entidad-Relación (Modelo de Estrella) estructurado para el análisis.

## 3. Enlace a Repositorio GitHub.

**Repositorio GitHub**

## 4. Enlace Google Drive

**DATA DRIVE**

# 5. Estimación Teórica y Proyección de Espacio en Disco.

## 5.1. 1. Tamaño por tipo de dato y fila (Row Size)

Para realizar la estimación de espacio, se sumaron los bytes requeridos por cada columna de las tablas principales (`viajes`, `historico_viajes`, `dim_usuarios`, `estaciones`, etc.), utilizando los pesos estándar de MySQL: `INT` = 4 bytes, `DATETIME` = 5 bytes, `SMALLINT` = 2 bytes y `TINYINT` = 1 byte.

A esto se sumó la sobrecarga (*overhead*) interna de InnoDB: encabezado de fila (∼5 bytes), ID de transacción (6 bytes) y puntero de rollback (7 bytes). Considerando el tamaño por defecto de la página InnoDB (16 KB) y un *fill factor* de 15/16, se determinó que la fila promedio ocupa aproximadamente **70 bytes** (incluyendo índices B-Tree de las Claves Primarias y Foráneas).

## 5.2. 2. Cálculo del Espacio de Carga Inicial

Con la fila promedio de 70 bytes, se proyectó la carga inicial antes de la inyección física de datos:

- **Carga Inicial de `historico_viajes`** (∼30,000,000 registros): ∼2.1 GB de datos + ∼1.2 GB de índices = ∼**3.3 GB**.
- **Carga Inicial de `viajes`** (∼6,500,000 registros): ∼455 MB de datos + ∼260 MB de índices = ∼**715 MB**.
- **Catálogos y Dimensiones:** ∼**15 MB**.
- **Total Carga Inicial:** ∼**4.03 GB**.

## 5.3. 3. Simulación y Proyección de Crecimiento

Asumiendo un promedio de 800,000 viajes mensuales nuevos en la CDMX y un incremento anual estimado del 10 % debido a nuevas estaciones. El crecimiento mensual proyectado es: 800,000 × 100 bytes (datos + índices) ≈ 80 MB/mes (lo que equivale a ∼0.96 GB/año).

| **Periodo** | **Almacenamiento Estimado** |
|---|---|
| Año 1 (Actual / Carga Inicial) | 4.03 GB |
| Año 2 (+10 % uso) | ∼5.08 GB |
| Año 3 (+20 % uso) | ∼6.24 GB |
| Año 5 (+40 % uso) | ∼8.95 GB |

Tabla 2: Tabla de Proyección a 5 Años

# 6. Consulta al Diccionario de Datos y Comparación de Almacenamiento.

## 6.1. Justificación Teórica

En MySQL 8.0, los metadatos se almacenan de forma centralizada y transaccional en el Diccionario de Datos. Medir los archivos físicos (como `.ibd`) directamente desde el sistema operativo resulta impreciso debido a dos factores:

- **Asignación por Extents:** El motor InnoDB reserva espacio en bloques completos, por lo que el archivo en disco siempre será más grande que los datos reales almacenados.
- **Fragmentación:** El espacio de los registros eliminados no se devuelve inmediatamente al sistema operativo.

Por ello, consultar el `INFORMATION_SCHEMA` permite al administrador medir el espacio lógico real, exacto y en uso.

## 6.2. Desarrollo de Consultas SQL

### A. Tamaño total de la Base de Datos

```
    SELECT table_schema AS Base_de_Datos,
    ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS Tamano_MB
    FROM information_schema.tables
    WHERE table_schema = 'DB_Ecobici_CDMX'
    GROUP BY table_schema;
```

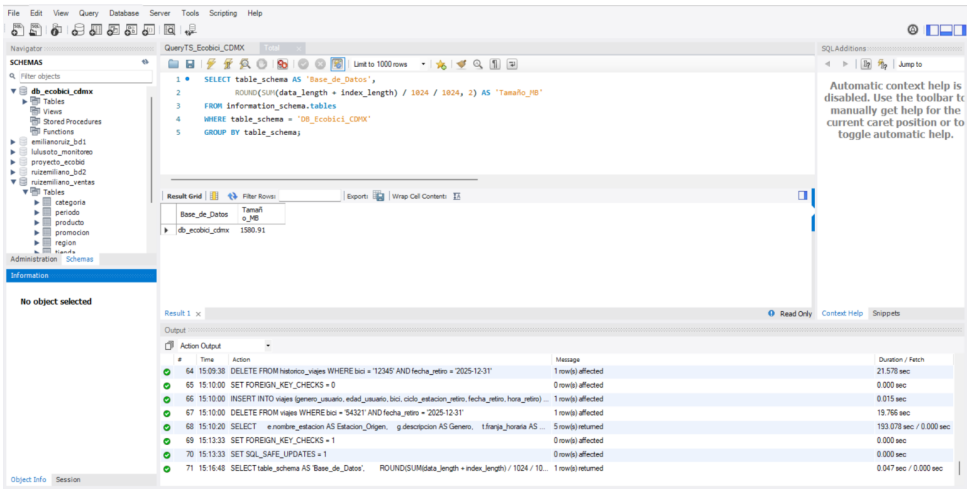



Figura 2: Tamaño total de la base de datos.

Como se observa en el resultado, el tamaño lógico total del esquema `DB_Ecobici_CDMX` es de **1580.91 MB** (aproximadamente 1.58 GB). Este valor representa la suma exacta de los datos puros más el peso de los índices, confirmando que la carga masiva de los registros se completó exitosamente.

### B. Tamaño individual por tabla e índice

```
SELECT table_name AS `Tabla`,
       ROUND(data_length / 1024 / 1024, 2) AS `Datos_MB`,
       ROUND(index_length / 1024 / 1024, 2) AS `Indices_MB`,
       table_rows AS `Numero_Registros`
FROM information_schema.tables
WHERE table_schema = 'DB_Ecobici_CDMX'
ORDER BY (data_length + index_length) DESC;
```

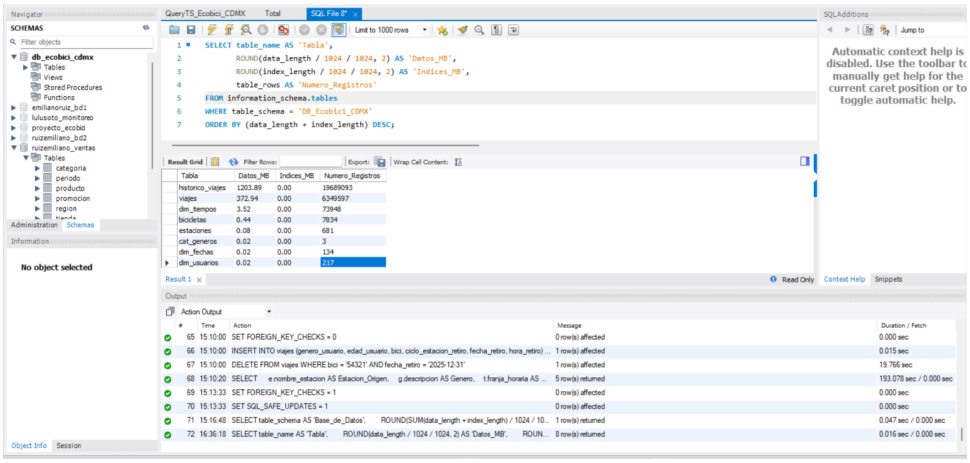


Figura 3: Desglose de peso por datos e índices de cada tabla.

### C. Consulta de Tablespaces y archivos .ibd

```sql
SELECT TABLESPACE_NAME AS `Particion_Tablespace`,
       FILE_NAME AS `Ruta_Archivo_ibd`,
       ROUND(TOTAL_EXTENTS * EXTENT_SIZE / 1024 / 1024, 2) AS `Espacio_Asignado_MB`,
       ROUND(FREE_EXTENTS * EXTENT_SIZE / 1024 / 1024, 2) AS `Espacio_Libre_MB`
FROM information_schema.FILES
WHERE TABLESPACE_NAME LIKE 'DB_Ecobici_CDMX/historico_viajes%';
```

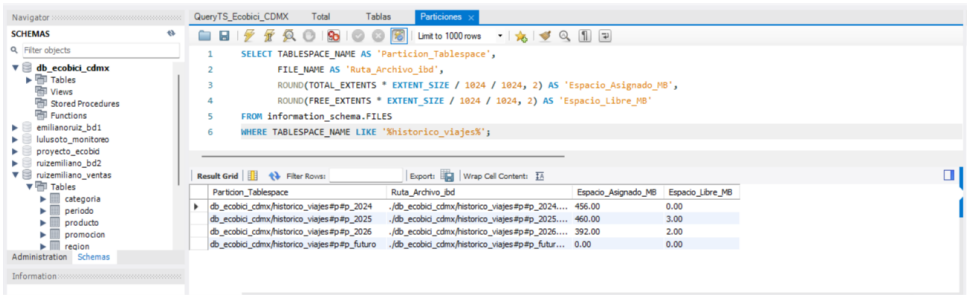


Figura 4: Espacio físico asignado vs libre en las particiones de la tabla histórica.

## 6.3. Matriz de Comparación

El siguiente cuadro contrasta la estimación matemática contra la medición real del Diccionario de Datos.

| Tabla | Filas (Aprox) | Teórico (MB) | Real (MB) | Diferencia |
|---|---|---|---|---|
| historico_viajes | 19,689,093 | 3379.20 | 1203.89 | -2175.31 |
| viajes | 6,349,597 | 715.00 | 372.94 | -342.06 |
| Catálogos (Suma) | 82,683 | 15.00 | 4.10 | -10.90 |

Tabla 3: Comparativa Estimación Teórica vs Diccionario de Datos

**Análisis:** El tamaño real resultó menor a la estimación teórica principalmente porque el modelo matemático calculó la tabla histórica sobre 30 millones de registros, mientras que la carga física procesó aproximadamente 20 millones.

# 7. Alta Disponibilidad, Replicación y Casos de Failover.

## 7.1. Servidores Primario, Réplica y Balanceador usando Docker.

### 7.1.1. Creando el entorno

Terminal de WSL (Ubuntu). Crear la estructura de carpetas exacta que necesitamos.

Listing 1: Creación de la estructura de directorios

```
mkdir -p ~/proyecto_ecobici/haproxy \
         ~/proyecto_ecobici/master/conf/mysql.conf.d \
         ~/proyecto_ecobici/slave/conf/mysql.conf.d
```

```
cd ~/proyecto_ecobici
```

### 7.1.2. Visual Studio Code

En un nuevo archivo en la raíz de la carpeta llámado docker-compose.yml.

### 7.1.3. Crea el archivo del Balanceador de Carga

En VS Code, en la carpeta haproxy, se crea un archivo llamado haproxy.cfg.
Terminal de terminal de WSL en caso se problemas por caracteres

```
    cat << 'EOF' > haproxy/haproxy.cfg
global
    log stdout format raw local0
    maxconn 1000

defaults
    log     global
    mode    tcp
    option  tcplog
    timeout connect 5s
    timeout client  300s
    timeout server  300s

frontend mysql_frontend
    bind *:3306
    default_backend mysql_backend

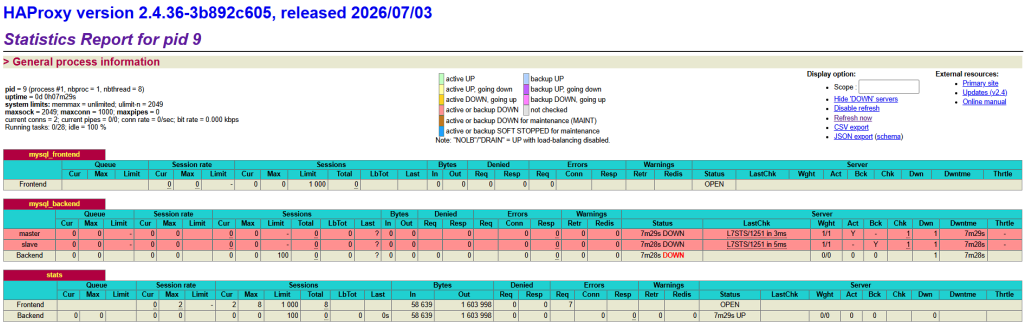
backend mysql_backend
    mode tcp
    option mysql-check user root
    server master mysql-master:3306 check inter 2000 rise 2 fall 3
    server slave mysql-slave:3306 check backup inter 2000 rise 2 fall 3

listen stats
    bind *:8404
    mode http
    stats enable
    stats uri /stats
    stats refresh 5s
EOF
```

### 7.1.4. Iniciar los servidores

```
    docker compose up -d
```

```
    http://localhost:8404/stats.
```



**HAProxy version 2.4.36-3b892c605, released 2026/07/03**

*Statistics Report for pid 9*

**> General process information**

**pid** = 9 (process #1, nbproc = 1, nbthread = 8)
**uptime** = 0d 0h07m29s
**system limits:** memmax = unlimited; ulimit-n = 2049
**maxsock** = 2049; **maxconn** = 1000; **maxpipes** = 0
current conns = 2; current pipes = 0/0; conn rate = 0/sec; bit rate = 0.000 kbps
Running tasks: 0/28; idle = 100 %

active UP
active UP, going down
active DOWN, going up
active or backup DOWN
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance
backup UP
backup UP, going down
backup DOWN, going up
not checked
Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

**Display option:**
- Scope:
- Hide 'DOWN' servers
- Disable refresh
- Refresh now
- CSV export
- JSON export (schema)

**External resources:**
- Primary site
- Updates (v2.4)
- Online manual

**mysql_frontend**

| | Queue | | | Session rate | | | Sessions | | | | | | Bytes | | Denied | | Errors | | Warnings | | Server | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Cur | Max | Limit | Cur | Max | Limit | Cur | Max | Limit | Total | LbTot | Last | In | Out | Req | Resp | Req | Conn | Resp | Retr | Redis | Status | LastChk | Wght | Act | Bck | Chk | Dwn | Dwntme | Thrtle |
| Frontend | | | | 0 | 0 | - | 0 | 0 | 1 000 | 0 | | | 0 | 0 | 0 | 0 | 0 | | | | | OPEN | | | | | | | |

**mysql_backend**

| | Queue | | | Session rate | | | Sessions | | | | | | Bytes | | Denied | | Errors | | | Warnings | | Server | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Cur | Max | Limit | Cur | Max | Limit | Cur | Max | Limit | Total | LbTot | Last | In | Out | Req | Resp | Req | Conn | Resp | Retr | Redis | Status | LastChk | Wght | Act | Bck | Chk | Dwn | Dwntme | Thrtle |
| master | 0 | 0 | - | 0 | 0 | | 0 | 0 | - | 0 | 0 | ? | 0 | 0 | | 0 | | 0 | 0 | 0 | 0 | 7m29s DOWN | L7STS\1251 in 3ms | 1/1 | Y | - | 1 | 1 | 7m29s | - |
| slave | 0 | 0 | - | 0 | 0 | | 0 | 0 | - | 0 | 0 | ? | 0 | 0 | | 0 | | 0 | 0 | 0 | 0 | 7m29s DOWN | L7STS\1251 in 5ms | 1/1 | - | Y | 1 | 1 | 7m29s | - |
| Backend | 0 | 0 | | 0 | 0 | | 0 | 0 | 100 | 0 | 0 | ? | 0 | 0 | 0 | 0 | | 0 | 0 | 0 | 0 | 7m29s DOWN | | 0/0 | 0 | 0 | | 1 | 7m29s | |

**stats**

| | Queue | | | Session rate | | | Sessions | | | | | | Bytes | | Denied | | Errors | | Warnings | | Server | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Cur | Max | Limit | Cur | Max | Limit | Cur | Max | Limit | Total | LbTot | Last | In | Out | Req | Resp | Req | Conn | Resp | Retr | Redis | Status | LastChk | Wght | Act | Bck | Chk | Dwn | Dwntme | Thrtle |
| Frontend | | | | 0 | 2 | - | 2 | 8 | 1 000 | 8 | | | 58 639 | 1 603 998 | 0 | 0 | 7 | | | | | OPEN | | | | | | | |
| Backend | 0 | 0 | | 0 | 0 | | 0 | 0 | 100 | 0 | 0 | 0s | 58 639 | 1 603 998 | 0 | 0 | | 0 | 0 | 0 | 0 | 7m29s UP | | 0/0 | 0 | 0 | | 0 | | |

Figura 5: Localhost

## 7.2. Configuración de Permisos para HAProxy (Health Checks)

Se creó un usuario dedicado sin contraseña y sin privilegios, cuyo único propósito es responder al ping del balanceador.

**1. Actualizar la configuración de HAProxy** En el archivo `haproxy.cfg`, se sustituyó el usuario `root`:

```
    option mysql-check user root

    option mysql-check user haproxy_check
```

**2. Crear el usuario en los motores MySQL** Debido a incompatibilidades de encriptación entre HAProxy y MySQL 8.0, se forzó el método de autenticación `mysql_native_password` al crear el usuario en ambos contenedores.

*Para el Maestro:*

```
docker exec -it mysql-master mysql -uroot -p-e "CREATE USER 'haproxy_check'@'%'
    IDENTIFIED WITH mysql_native_password BY ''; FLUSH PRIVILEGES;"
```

*Para la Réplica:*

```
docker exec -it mysql-slave mysql -uroot -p -e "CREATE USER 'haproxy_check'@'%'
    IDENTIFIED WITH mysql_native_password BY ''; FLUSH PRIVILEGES;"
```

**3. Aplicar los cambios en el Balanceador** Se reinició el servicio para que HAProxy adoptara el nuevo usuario:

```
docker compose restart haproxy
```

Tras la ejecución, el panel de estadísticas en `http://localhost:8404/stats` confirmó que ambos nodos pasaron a estado activo (UP).

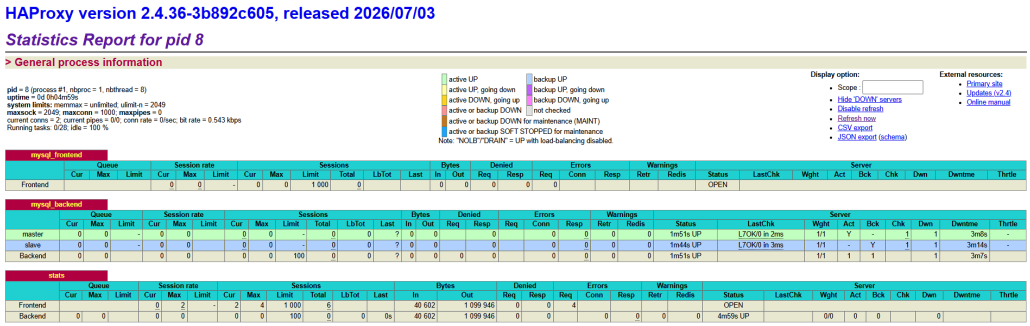



Figura 6: Localhost acceso a la base de datos (Estado UP)

## 7.3. Configuración de Replicación Maestro-Esclavo (GTID)

Para el cumplimiento del diseño de Alta Disponibilidad, se implementó replicación asíncrona mediante GTID (Global Transaction Identifier). GTID es el estándar moderno en MySQL que facilita la sincronización, ya que cada transacción recibe un ID único y la réplica sabe exactamente qué eventos procesar sin requerir posiciones exactas de archivos `binlog`.

### 7.3.1. Habilitar GTID y Replicación (Inyección vía Docker Compose)

Debido a restricciones de seguridad en los permisos de volúmenes compartidos entre Windows y WSL (donde el motor ignora archivos de configuración con permisos demasiado abiertos), se optó por prescindir de los archivos físicos `mysqld.cnf`.

En su lugar, las variables de entorno para habilitar GTID se inyectaron directamente en la definición de cada contenedor dentro del archivo `docker-compose.yml` utilizando la directiva `command`.

**Configuración del Maestro en docker-compose.yml**

```
    command: --server-id=1 --log-bin=mysql-bin --binlog-format=ROW --gtid-mode=ON --
    enforce-gtid-consistency=ON
```

**Configuración de la Réplica en docker-compose.yml**

```
    command: --server-id=2 --gtid-mode=ON --enforce-gtid-consistency=ON --read-only=1
```

### 7.3.2. Ejecución de la Arquitectura Limpia

Para purgar configuraciones residuales y metadatos corruptos, se procedió a destruir el entorno anterior (incluyendo los volúmenes físicos de datos con el flag `-v`) y a instanciar la infraestructura en limpio:

```
docker compose down -v
docker compose up -d
```

### 7.3.3. Comandos de Replicación SQL

**1. Obtener la IP interna del servidor Maestro** La Réplica requiere enlazarse directamente a la dirección IP privada del contenedor Maestro dentro de la red virtual de Docker (`ecobici-net`).

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mysql-master
```

*Resultado:* `172.18.xxx`

**2. Crear el usuario de replicación en el Maestro**

```
docker exec -it mysql-master mysql -uroot -p_xxxx
```

```
CREATE USER 'replica_user'@'%' IDENTIFIED WITH mysql_native_password BY 'xxx';
GRANT REPLICATION SLAVE ON *.* TO 'replica_user'@'%';
FLUSH PRIVILEGES;
EXIT;
```

**3. Conectar la Réplica al Maestro**

```
docker exec -it mysql-slave mysql -uroot -p_xxx
```

```
CHANGE MASTER TO
  MASTER_HOST='xxx',
  MASTER_USER='replica_user',
  MASTER_PASSWORD='xxx',
  MASTER_AUTO_POSITION=1;
START SLAVE;
```

### 7.3.4. Resolución de Incidentes: Error 1396 y Conflicto GTID

Durante el proceso de conexión (`START SLAVE;`), el *Coordinator worker* detuvo la clonación arrojando el error `Last_SQL_Errno: 1396` (`Operation CREATE USER failed`). Esto ocurrió porque el servidor esclavo intentó replicar una transacción DDL de creación de usuario que ya existía físicamente en su propio disco.

Para desatascar el flujo sin reconstruir el entorno, se aplicó una técnica avanzada de DBA inyectando un GTID vacío correspondiente a la transacción conflictiva, instruyendo al motor a omitirla y continuar con la cola de eventos.

```
STOP SLAVE;
SET GTID_NEXT="a8d9d588-cea3-11f1-ad19-0a3abf4f4181:7";
BEGIN;
COMMIT;
SET GTID_NEXT="AUTOMATIC";
START SLAVE;
```

Tras la corrección, la validación final mediante `SHOW SLAVE STATUS\G` confirmó la sincronización exitosa de ambos hilos:

- `Slave_IO_Running: Yes`
- `Slave_SQL_Running: Yes`

## 7.4. Cargar base de datos ECOBICI

El contenedor mysql-master es un sistema operativo Linux aislado. No tiene disco C: ni identifica la ruta:

```
C:\ProgramData\MySQL\MySQL Server 8.0\Uploads.
```

para lo LOAD DATA INFILE del script.

Pasaremos los archivos de Windows al entorno de Linux en Docker y se modificaran las rutas.

### 7.4.1. Copiar los archivos CSV al contenedor Linux

```
docker cp "/mnt/c/ProgramData/MySQL/MySQL Server 8.0/Uploads/." mysql-master:/var/
lib/mysql-files/
```

### 7.4.2. Mover y modificar el script SQL

Mover el archivo `QueryTS_Ecobici_CDMX.sql` a la carpeta del proyecto en Linux.

```
\\wsl$\Ubuntu\home\ricardo\proyecto_ecobici.
```

```
Buscar y Reemplazar
Buscar
C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/
Reemplazar
/var/lib/mysql-files/
```

### 7.4.3. Inyectar la base de datos

En la terminal de Linux ejecuta el comando para inyectar todo el script directamente al maestro:

```
docker exec -i mysql-master mysql -uroot -p_xxxx DB_Ecobici_CDMX <
QueryTS_Ecobici_CDMX.sql
```

**Importante**

Como este script carga más de 6 millones de registros y construye llaves foráneas e índices, el comando se quedará pensando durante unos 15 a 20 minutos. No lo canceles.

Si quieres ver cómo tu computadora está trabajando, puedes abrir una nueva pestaña o ventana de terminal en WSL y ejecutar el comando. Ahí verás en tiempo real cómo el contenedor mysql-master está utilizando al máximo tu CPU y memoria RAM.

```
docker stats.
```

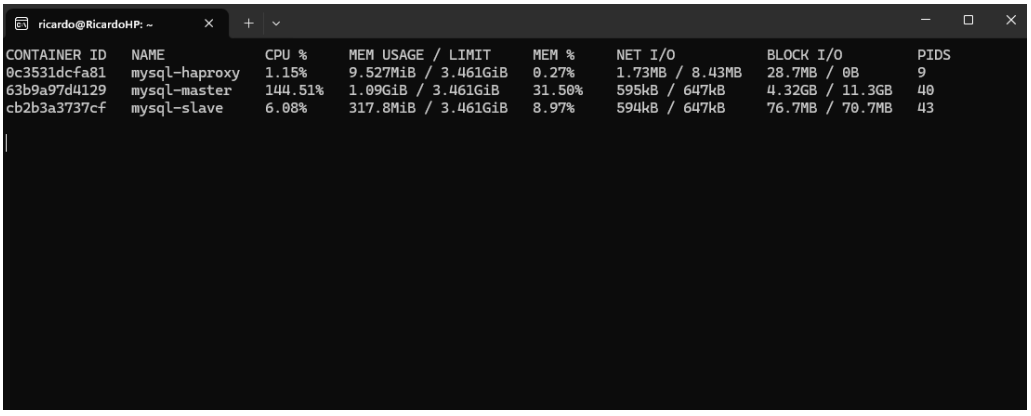



Figura 7: Estatus carga

# 8. Crear Cuenta Google Cloud

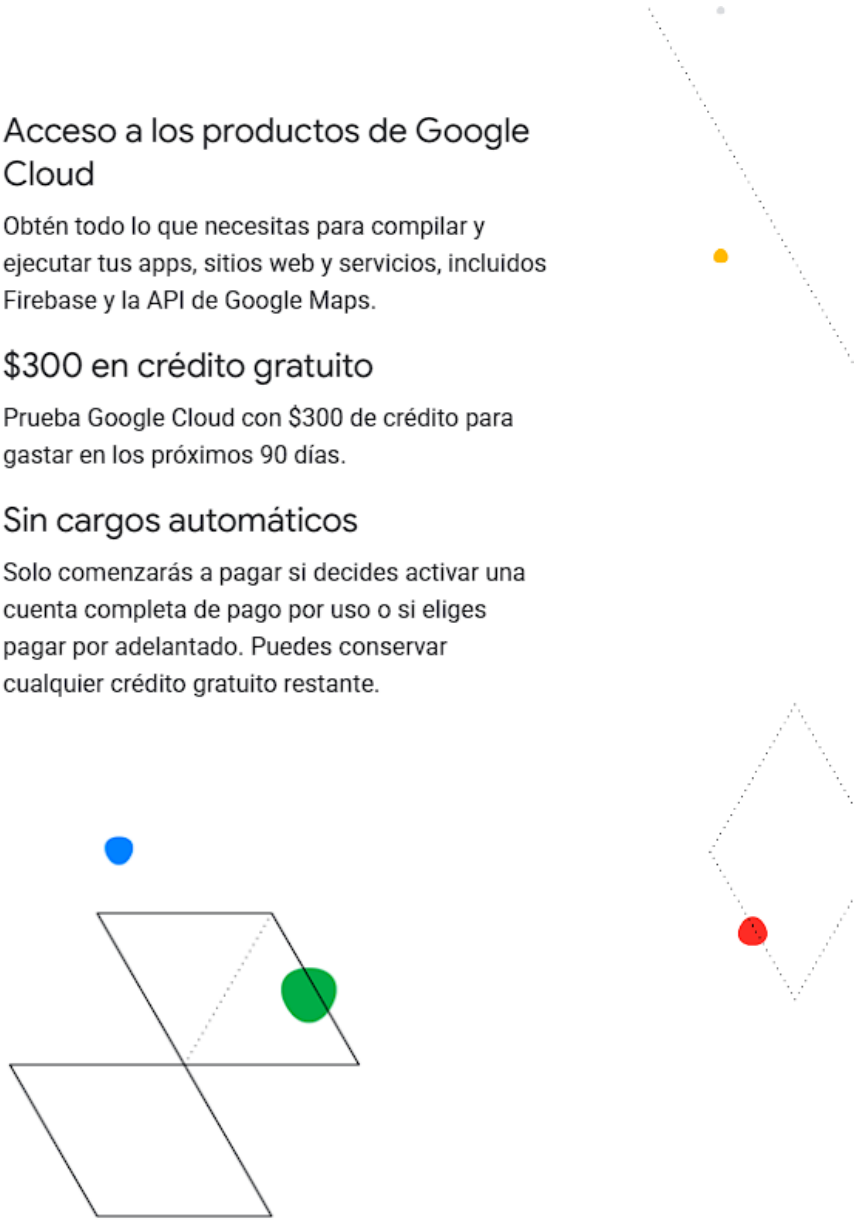



Figura 8: GCP

## 8.1. Crear Instancias

- Ingresa a la Consola de Google Cloud.
- En el menú de navegación ve a Compute Engine >Instancias de VM.
- Haz clic en Crear instancia (Create instance).
- Configura los siguientes parámetros: Nombre: nodo-respaldo-ecobici Región: Selecciona us-central1 (Iowa) Tipo de máquina: Propósito general, serie E2, selecciona e2-micro (Es la opción más barata).

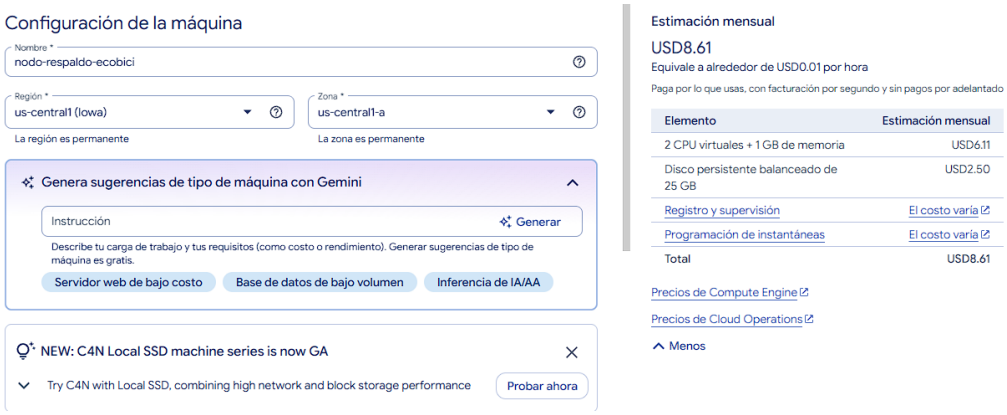



Figura 9: Cofg mv

- Disco de arranque (Boot disk): Haz clic en Cambiar. Sistema Operativo: Selecciona Ubuntu. Versión: Selecciona Ubuntu 24.04 LTS.

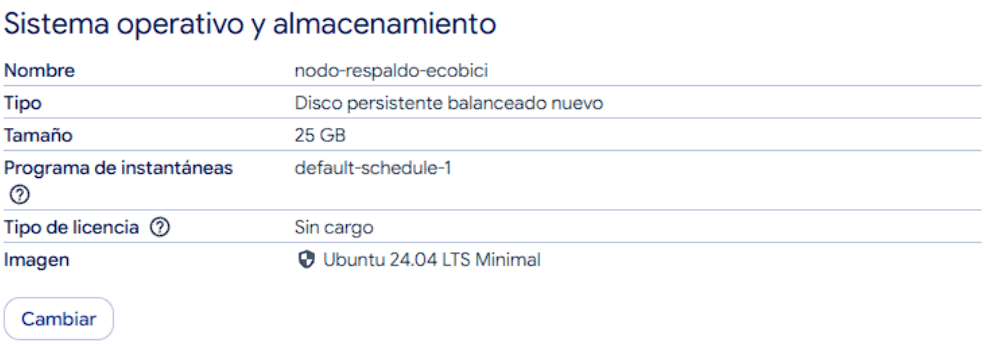



Figura 10: Cof SO

Tamaño: 10 GB (por defecto). Firewall: Marca la casilla Permitir tráfico HTTP y Permitir tráfico HTTPS.

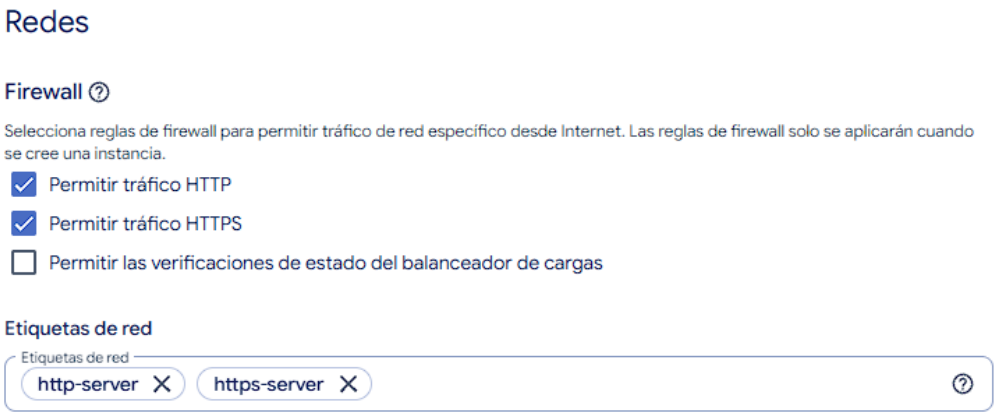



Figura 11: Firewall

Haz clic en el botón azul Crear al final de la página.

### 8.1.1. Regla de Firewall adicional

Las casillas HTTP/HTTPS que se marcaron abren los puertos 80 y 443. Pero necesitamos abrir los puertos específicos de la bases de datos (3306 y 8404).

- En el submenú que se despliega, haz clic en Firewall.
- En la parte superior central de la pantalla de Firewall, busca y haz clic en el botón que dice CREAR REGLA DE FIREWALL (Create Firewall Rule).
- Nombre: allow-haproxy
- Dirección del tráfico: Entrada
- Acción: Permitir
- Destinos: Todas las instancias de la red
- Rangos IPv4 de origen: 0.0.0.0/0
- Puertos TCP: 3306,8404

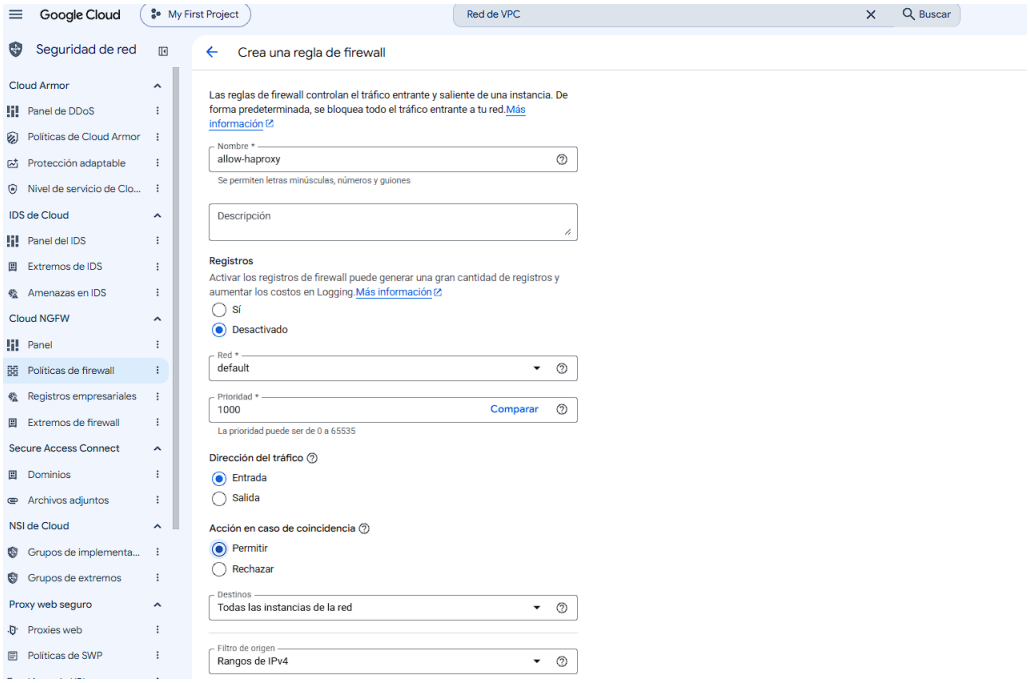



Figura 12: Firewall

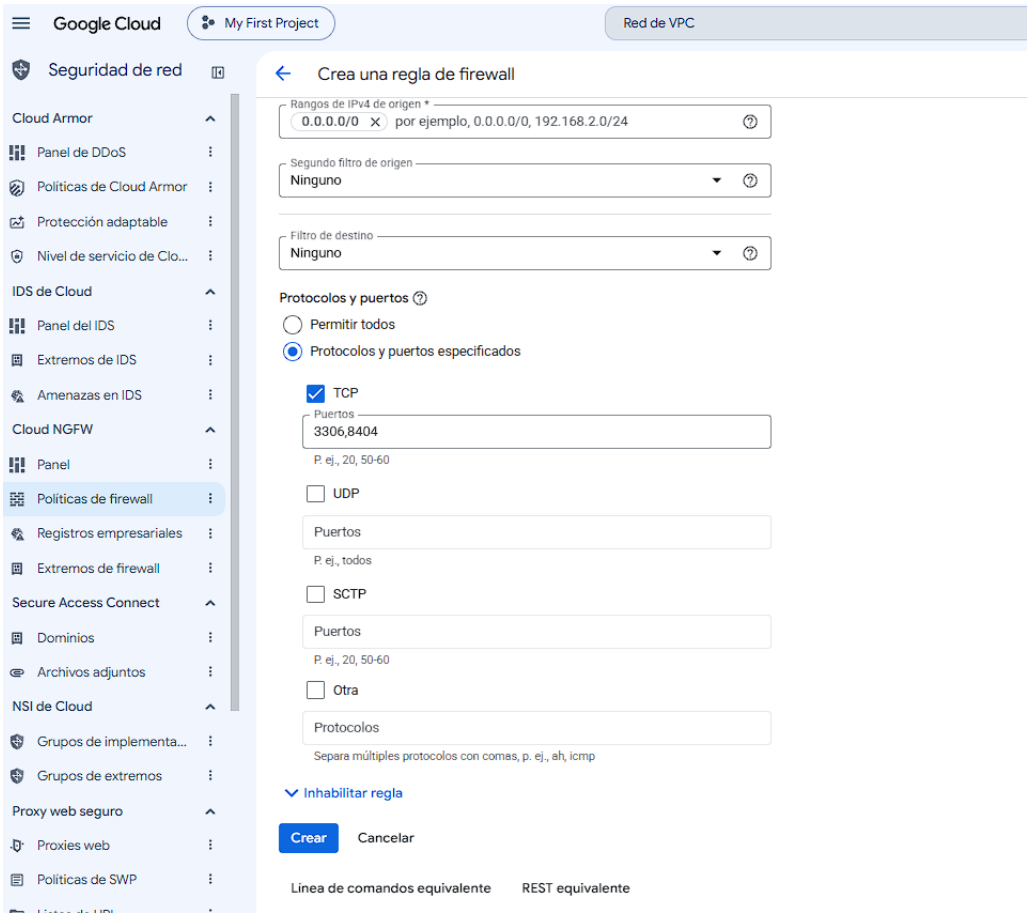



Figura 13: Firewall

### 8.1.2. Maquina Virtual

Maquina Virtual: Se observa la terminal de Ubuntu 2404 LTS conectada vá SSH

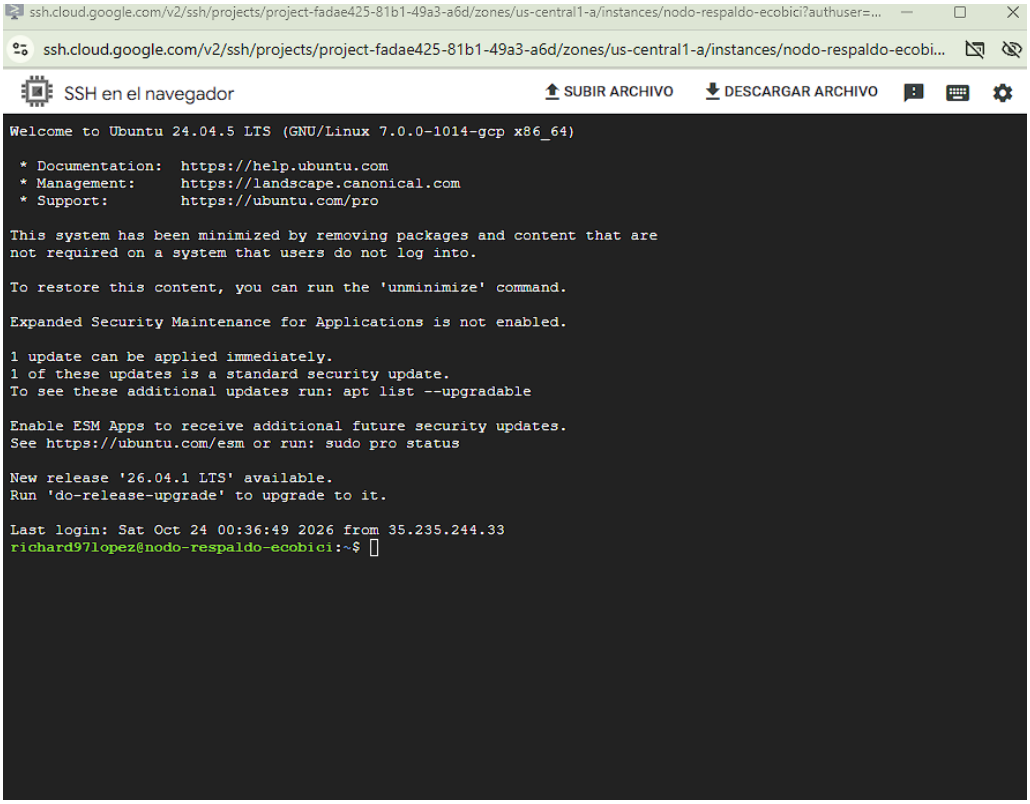



Figura 14: Firewall

### 8.1.3. Instalar Docker y Docker Compose

Se instalara Docker en la maquina virtual y se caragra el proyecto
Primero actualizaremos los paquetes del sistema y luego instalaremos el motor de Docker.

```
sudo apt update && sudo apt upgrade -y
sudo apt install docker.io docker-compose-v2 -y
```

### 8.1.4. Dar permisos a tu usuario

Para no tener que escribir sudo antes de cada comando de Docker, añadiremos tu usuario al grupo de Docker:

```
sudo usermod -aG docker $USER
newgrp docker
```

### 8.1.5. Crear la estructura del proyecto

Crearemos las mismas carpetas que estan el localhost.

```
mkdir -p ~/proyecto_ecobici/haproxy
cd ~/proyecto_ecobici
```

### 8.1.6. Instalacion editor de texto nano

```
sudo apt install nano -y
```

### 8.1.7. Crear docker-compose.yml

Arquitectura con GTID inyectado.

```
nano docker-compose.yml
```

### 8.1.8. Crear haproxy.cfg

La configuración del balanceador de carga dentro de su respectiva carpeta.

```
nano haproxy/haproxy.cfg
```

### 8.1.9. Arrancar la infraestructura en la nube

Con los archivos listos, ejecuta este comando para descargar las imágenes y encender los servidores de base de datos.

```
docker compose up -d
```

La arquitectura esta completa.

- Maestro
- Esclavo
- HAProxy

Estan levantados los contenedores y de forma estable en la maquina virtual de GCP.

### 8.1.10. Configuración de Permisos para HAProxy (Health Checks)

La infraestructura funciona, los nodos de MySQL rechazan la conexión del balanceador porque todavía no se crea el usuario que HAProxy usa para verificar la salud de los servidores haproxy check.
Para el Maestro:

```
docker exec -it mysql-master mysql -uroot -p_ -e "DROP USER IF EXISTS 'haproxy_check
'@'%'; CREATE USER 'haproxy_check'@'%' IDENTIFIED WITH mysql_native_password BY '';
FLUSH PRIVILEGES;"
```

Para la Réplica:

```
docker exec -it mysql-slave mysql -uroot -p_ -e "DROP USER IF EXISTS 'haproxy_check'@
'%'; CREATE USER 'haproxy_check'@'%' IDENTIFIED WITH mysql_native_password BY '';
FLUSH PRIVILEGES;"
```

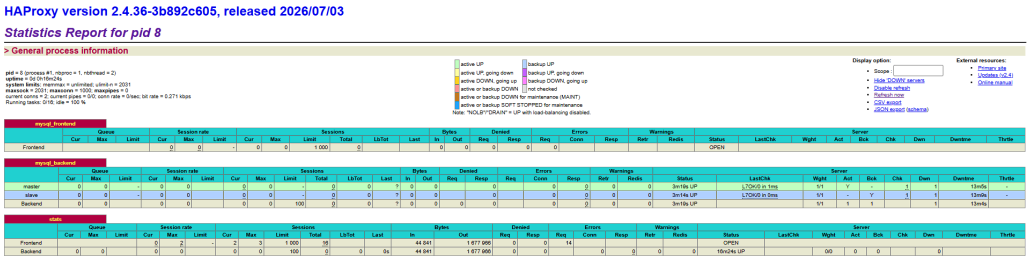



Figura 15: HAProxy

### 8.1.11. Replicacion GTID

IP interna del Maestro

```
docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' mysql-
 master
```

### 8.1.12. Crear el usuario de clonación en el Maestro

Entra a la terminal del Maestro:

```
docker exec -it mysql-master mysql -uroot -p_XXX
```

MySQL.

```
 CREATE USER 'replica_user'@'%' IDENTIFIED WITH mysql_native_password BY 'xxx';
 GRANT REPLICATION SLAVE ON *.* TO 'replica_user'@'%';
 FLUSH PRIVILEGES;
 EXIT;
```

Conectar la Réplica al Maestro:

```
docker exec -it mysql-slave mysql -uroot -p_xxxx
```

MySQL.

```
 STOP SLAVE;

 CHANGE MASTER TO
 MASTER_HOST='mysql-master',
 MASTER_USER='replica_user',
 MASTER_PASSWORD='xxxx',
 MASTER_AUTO_POSITION=1;

 START SLAVE;

 SHOW SLAVE STATUS\G
```

### 8.1.13. Comprobar la conexión

```
SHOW SLAVE STATUS\G
```

La replicación asíncrona por GTID está conectada y funcionando de forma estable.

Se contruyo, desplego y estabilizo una arquitectura de Alta Disponibilidad Master - Slave con balanceador de carga en HAProxy en GCP.

# 9. Poblar la Base de Datos

Necesitas enviar el respaldo de tu esquema relacional local a la nube. abriendo MySQL Workbench, localhost y creando una nueva conexión hacia tu instancia en GCP.

- Host: 34.122.XXXX
- Puerto: XXXX.
- Contraseña: XXXX.

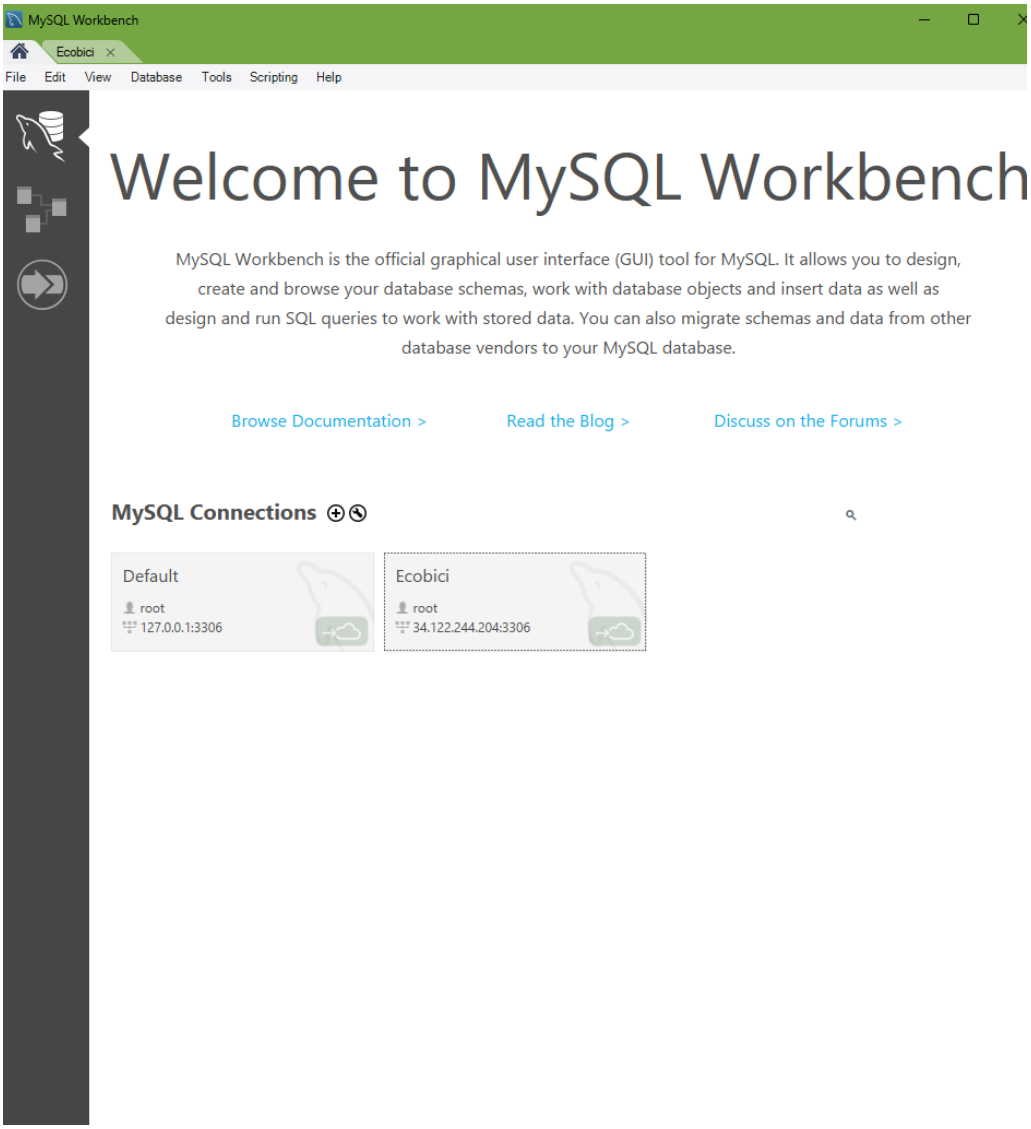



Figura 16: HAProxy

## 9.1. Cargar Data en la Maquina Virtual GCP

Cargar la Data de Ecobici suben 12 archivos CSV a la nube.

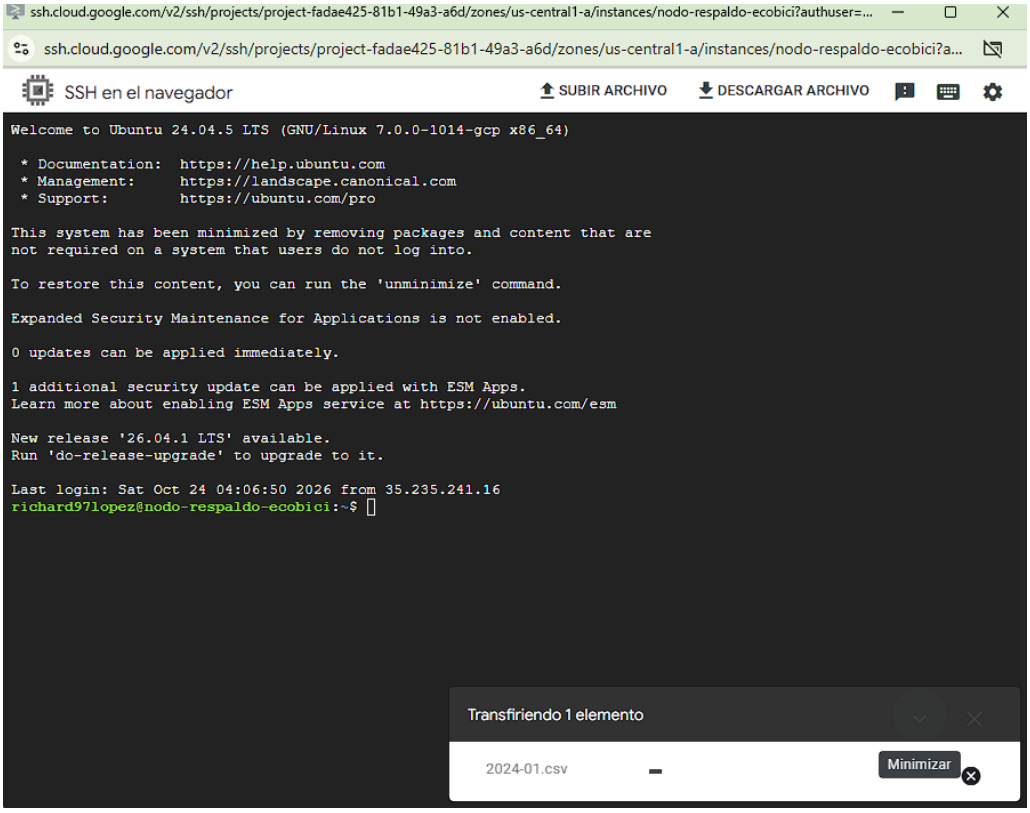



Figura 17: Data

### 9.1.1. Copiar dentro del contenedor Maestro

Sedeben copiar dentro del contenedor Maestro usando el comando en tu terminal SSH:

```
docker cp *.csv mysql-master:/var/lib/mysql-files/
```

Finalmente, en MySQL Workbench, modifica todas las rutas de los archivos en tu script.

```
Cambia
C:/ProgramData/MySQL/
por
/var/lib/mysql-files/nombre_del_archivo.csv.
```

Los 12 Archios CSV ya están alojados dentro del nodo Maestro en la ruta

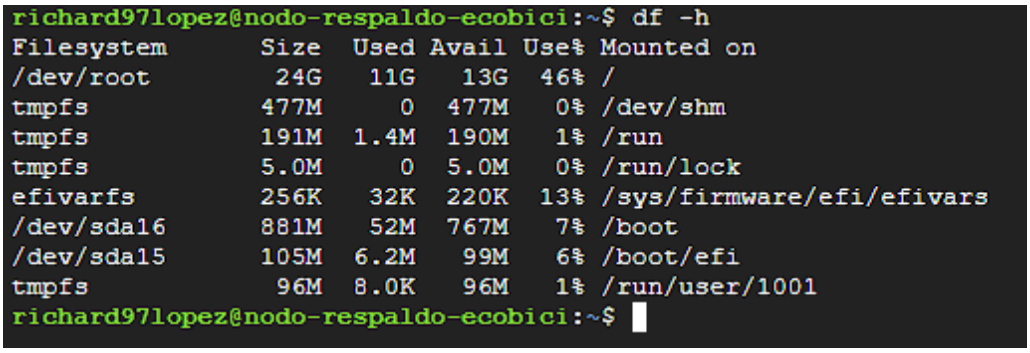



Figura 18: VM

### 9.1.2. Ejecución del Script en MySQL Workbench

- Script QueryTS Ecobici CDMX en MySQL Workbench.
- Herramienta de Buscar y Reemplazar.
- Escribe la ruta exacta de Windows que tiene tu script actualmente.

```
        C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/

```

- Replace, escribe la nueva ruta de Linux.

```
        /var/lib/mysql-files/

```

- Ejecutar todo el script

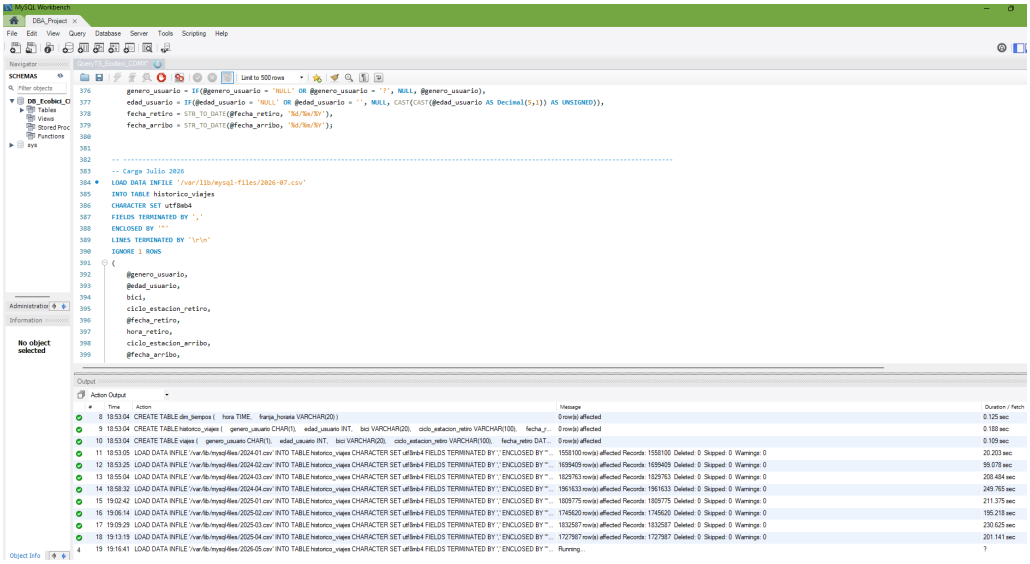


Figura 19: VM

### 9.1.3. PRIMARY KEY

Se agregan las llaves primarias.

```
    ALTER TABLE cat_generos ADD PRIMARY KEY (codigo_genero);
    ALTER TABLE estaciones ADD PRIMARY KEY (id_estacion);
    ALTER TABLE bicicletas ADD PRIMARY KEY (bici);
    ALTER TABLE dim_usuarios ADD PRIMARY KEY (id_perfil);
    ALTER TABLE dim_fechas ADD PRIMARY KEY (fecha);
    ALTER TABLE dim_tiempos ADD PRIMARY KEY (hora);
```

### 9.1.4. FOREIGN KEY

```
    ALTER TABLE viajes
        ADD CONSTRAINT fk_viajes_genero FOREIGN KEY (genero_usuario) REFERENCES
    cat_generos(codigo_genero),
        ADD CONSTRAINT fk_viajes_bici FOREIGN KEY (bici) REFERENCES bicicletas(bici),
        ADD CONSTRAINT fk_viajes_estacion_retiro FOREIGN KEY (ciclo_estacion_retiro)
    REFERENCES estaciones(id_estacion),
        ADD CONSTRAINT fk_viajes_estacion_arribo FOREIGN KEY (ciclo_estacion_arribo)
    REFERENCES estaciones(id_estacion),
        ADD CONSTRAINT fk_viajes_fecha_retiro FOREIGN KEY (fecha_retiro) REFERENCES
    dim_fechas(fecha),
        ADD CONSTRAINT fk_viajes_hora_retiro FOREIGN KEY (hora_retiro) REFERENCES
    dim_tiempos(hora);
```